CSS — mise en forme et design web

Informatique — Licence 1 (L1)

Cours INF-L1-05

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Donner du style aux pages web : syntaxe CSS et sélecteurs, modèle de boîte, couleurs et typographie, puis les deux grandes techniques de mise en page modernes, Flexbox et CSS Grid, le design responsive et les animations — jusqu'à la reproduction complète d'une maquette.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Syntaxe CSS : sélecteurs, propriétés et valeurs
• Leçon 2 — Le modèle de boîte : marges, bordures et padding
• Leçon 3 — Couleurs, arrière-plans et typographie
• Leçon 4 — Flexbox : aligner et répartir les éléments
• Leçon 5 — CSS Grid : mises en page en deux dimensions
• Leçon 6 — Design responsive et media queries
• Leçon 7 — Transitions et animations CSS
• Leçon 8 — Projet : reproduire une maquette web moderne
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Le cours précédent t'a appris à structurer une page avec HTML : des titres, des paragraphes, des liens, des images. Mais une page HTML sans style ressemble à un document brut — texte noir sur fond blanc, tout aligné à gauche. Le CSS (Cascading Style Sheets, feuilles de style en cascade) est le langage qui transforme cette structure en une vraie page web : couleurs, espacements, colonnes, alignements, adaptation au téléphone. C'est lui qui décide de tout ce que l'œil voit, pendant que le HTML décide de ce que la page signifie.

Ce cours avance des fondations vers la pratique professionnelle. Tu commenceras par la syntaxe et les sélecteurs — savoir désigner précisément l'élément à habiller — puis par le modèle de boîte, le concept le plus important du CSS : chaque élément est une boîte rectangulaire entourée d'espaces et de bordures. Viendront ensuite les couleurs et la typographie, qui donnent son identité à un site, puis les deux systèmes de mise en page qui ont remplacé les anciennes astuces : Flexbox pour aligner des éléments sur un axe, et CSS Grid pour composer de vraies grilles en deux dimensions.

La fin du cours répond à la réalité d'aujourd'hui : la majorité des visiteurs naviguent sur téléphone. Tu apprendras le design responsive et les media queries, qui permettent à une même page de s'adapter à tous les écrans, puis les transitions et animations qui rendent une interface vivante sans la rendre lourde. La dernière leçon rassemble tout dans un projet : reproduire une maquette moderne de A à Z, avec une méthode que tu réutiliseras pour chacun de tes projets.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Écrire des règles CSS valides et cibler les éléments avec les bons sélecteurs (balise, classe, identifiant, descendants, pseudo-classes).
- Expliquer la cascade, la spécificité et l'héritage pour prévoir quel style s'applique à un élément.
- Maîtriser le modèle de boîte (contenu, padding, bordure, marge) et le rôle de box-sizing.
- Composer des mises en page flexibles avec Flexbox (alignements, répartition, retours à la ligne).
- Construire des grilles en deux dimensions avec CSS Grid (colonnes, lignes, zones nommées).
- Rendre une page responsive avec les media queries et une démarche « mobile first ».
- Ajouter transitions et animations CSS sobres, puis reproduire méthodiquement une maquette complète.

Leçon 1 — Syntaxe CSS : sélecteurs, propriétés et valeurs

Une règle CSS comporte deux parties : un sélecteur, qui désigne les éléments concernés, et un bloc de déclarations entre accolades. Chaque déclaration associe une propriété et une valeur, séparées par deux-points et terminées par un point-virgule : p { color: blue; font-size: 16px; } signifie « tous les paragraphes auront un texte bleu de 16 pixels ». Les commentaires s'écrivent entre /* et */ et sont ignorés par le navigateur : utilise-les pour organiser ta feuille de style en sections. Il existe trois façons d'appliquer du CSS : dans un fichier externe relié par <link> (la bonne pratique, car un seul fichier habille tout le site), dans une balise <style> dans l'en-tête de la page, ou directement dans l'attribut style d'un élément — à réserver aux dépannages, car impossible à maintenir.

Les sélecteurs de base sont au nombre de trois : la balise (p, h1, a) vise tous les éléments de ce type ; la classe, précédée d'un point (.carte), vise tous les éléments portant attribut class="carte" — c'est le sélecteur le plus utilisé, car réutilisable partout ; l'identifiant, précédé d'un dièse (#menu), vise un élément unique portant id="menu". On combine les sélecteurs : un espace désigne un descendant (nav a = les liens situés dans la navigation), et les pseudo-classes ajoutent un état ou une position, comme :hover (survol de la souris), :first-child (premier enfant) ou :nth-child(2) (deuxième enfant).

Quand plusieurs règles visent le même élément, le navigateur tranche grâce à la cascade et à la spécificité. En cas de conflit, la règle la plus spécifique gagne : un identifiant bat une classe, qui bat une balise ; à spécificité égale, c'est la dernière règle lue qui l'emporte. Certaines propriétés, comme color ou font-family, sont en plus héritées : un paragraphe reçoit la couleur définie sur son parent s'il n'a pas la sienne. Comprendre ces trois mécanismes — cascade, spécificité, héritage — évite la

frustration du « mon style ne s'applique pas » : presque toujours, une règle plus spécifique ou plus tardive écrase la tienne.

L'essentiel
Une règle CSS = un sélecteur + des déclarations propriété: valeur. Sélecteurs : balise, .classe (réutilisable), #identifiant (unique), descendants et pseudo-classes comme :hover. En cas de conflit : la spécificité décide (id > classe > balise), puis l'ordre d'apparition.

Exemple — Premières règles et pseudo-classe :hover

```
/* Feuille de style externe : styles.css */
body {
font-family: Arial, sans-serif;
color: #222222;
}

.titre-principal {
color: #0b3d91;
text-align: center;
}

nav a {
color: #ffffff;
text-decoration: none;
padding: 8px 12px;
}

nav a:hover {
background-color: #c9a227;
}

#logo {
font-weight: bold;
font-size: 24px;
}
```

À retenir

- Le CSS externe (fichier .css relié par <link>) est la norme : un fichier pour tout le site.
- .classe se réutilise sur autant d'éléments qu'on veut ; #identifiant ne doit exister qu'une fois par page.
- Spécificité : identifiant > classe > balise ; à égalité, la dernière règle gagne.

Exercice

Dans la page d'un club d'étudiants, on veut : 1) tous les titres h2 en bleu nuit ; 2) les boutons d'inscription (classe "btn-inscription", il y en a plusieurs) avec un fond doré ; 3) le titre principal de la page (identifiant "grand-titre") en très grand. Écris les trois règles CSS. Puis explique ce qui se passe si une quatrième règle, écrite plus bas, donne color: red au sélecteur h2.

✔ Corrigé
h2 { color: #0b2545; } — le sélecteur de balise vise tous les h2. .btn-inscription { background-color: #c9a227; } — la classe permet de styler tous les boutons d'un coup, présents et futurs. #grand-titre { font-size: 40px; } — l'identifiant ne concerne que le titre principal. Si une règle h2 { color: red; } apparaît plus bas dans le fichier, elle a la même spécificité que la première (une balise) : à spécificité égale, c'est la dernière lue qui gagne, donc tous les h2 deviendront rouges. Pour forcer le bleu nuit malgré tout, il faudrait un sélecteur plus spécifique (par exemple une classe .titre-section) — jamais le mot-clé !important, qui rend vite une feuille de style impossible à déboguer.

Leçon 2 — Le modèle de boîte : marges, bordures et padding

Le navigateur voit chaque élément HTML comme une boîte rectangulaire composée de quatre couches emboîtées. Au centre, le contenu (le texte ou l'image), dont on règle la taille avec width et height. Autour, le padding est l'espace intérieur, entre le contenu et la bordure : c'est lui qui « fait respirer » un bouton ou une carte. La bordure (border) entoure le tout, avec une épaisseur, un style (solid, dashed…) et une couleur. Enfin, la margin est l'espace extérieur, invisible, qui sépare la boîte de ses voisines. Retenir l'ordre de l'intérieur vers l'extérieur : contenu, padding, bordure, marge.

Ce modèle a une conséquence qui surprend tous les débutants : par défaut, width ne désigne que le contenu. Une boîte de width: 200px avec 20px de padding de chaque côté et 2px de bordure mesure en réalité 200 + 40 + 4 = 244 pixels à l'écran. La propriété box-sizing: border-box change ce calcul : width inclut alors le padding et la bordure, et 200px signifie 200 pixels au total. Les développeurs professionnels appliquent presque toujours * { box-sizing: border-box; } en tête de feuille de style pour raisonner simplement.

Deux comportements complètent le tableau. D'abord, les marges verticales de deux éléments voisins peuvent fusionner (margin collapsing) : si un paragraphe a margin-bottom: 30px et le suivant margin-top: 20px, l'écart réel sera de 30 pixels, pas 50 — le navigateur garde la plus grande des deux. Ensuite, la propriété display décide du comportement de la boîte : block (elle occupe toute la largeur et passe à la ligne, comme p ou div), inline (elle coule dans le texte, sans width ni height efficaces, comme a ou strong), ou inline-block (dans le texte, mais avec des dimensions réglables). Savoir cela évite de chercher pendant une heure pourquoi une marge « ne marche pas » sur un lien.

□ L'essentiel
Chaque élément = contenu + padding (intérieur) + border (bordure) + margin (extérieur). Par défaut, width ne couvre que le contenu ; box-sizing: border-box fait de width la largeur totale, ce qui simplifie toutes les mises en page.

Exemple — Une carte d'étudiant et ses espaces

```
* {
box-sizing: border-box;
}

.carte {
width: 300px;
padding: 20px; /* espace intérieur */
border: 2px solid #0b3d91; /* bordure bleue */
border-radius: 12px; /* coins arrondis */
margin: 16px auto; /* extérieur + centrage */
background-color: #ffffff;
}

/* Largeur totale réelle : 300px (padding et bordure inclus) */
```

À retenir

- Ordre des couches, de l'intérieur : contenu → padding → bordure → marge.
- margin: 0 auto centre horizontalement une boîte block de largeur définie.
- Les marges verticales voisines fusionnent : l'écart vaut la plus grande des deux, pas leur somme.

Exercice

Une boîte est définie ainsi : width: 240px; padding: 15px; border: 3px solid black; margin: 10px; (sans box-sizing particulier, donc en mode content-box par défaut). 1) Quelle est sa largeur totale occupée à l'écran, marges comprises ? 2) Même question si on ajoute box-sizing: border-box. 3) Pourquoi ce second mode est-il préféré pour construire des colonnes côte à côte ?

✔ Corrigé
1) En content-box : contenu 240 + padding gauche et droit 15 + 15 + bordures 3 + 3 = 276 pixels pour la boîte visible ; avec les marges de 10 de chaque côté, elle occupe 296 pixels dans la page. 2) En border-box : width inclut padding et bordure, la boîte visible mesure exactement 240 pixels (le contenu utile se réduit à 240 − 30 − 6 = 204 pixels), et 260 pixels marges comprises. 3) Pour des colonnes, on raisonne en parts du conteneur — par exemple deux colonnes de 50 %. En border-box, 50 % reste 50 % quel que soit le padding ajouté ensuite ; en content-box, chaque padding « déborde » et la deuxième colonne passe à la ligne, ce qui casse la mise en page dès qu'on retouche les espaces intérieurs.

Leçon 3 — Couleurs, arrière-plans et typographie

Le CSS offre plusieurs notations pour une même couleur. Les noms prédéfinis (red, navy, gold…) sont pratiques pour tester mais limités à environ 140 valeurs. La notation hexadécimale, #RRGGBB, est la plus répandue sur le web : #0b3d91 désigne un bleu composé de 0b de rouge, 3d de vert et 91 de bleu, chaque paire allant de 00 à ff. La notation rgb(11, 61, 145) donne le même résultat en décimal, et rgba ajoute un quatrième nombre, la transparence (0 = invisible, 1 = opaque) : rgba(11, 61, 145, 0.1) donne un bleu très pâle, idéal en fond de section. Enfin hsl(teinte, saturation,

luminosité) est souvent jugée plus intuitive pour créer des variantes d'une même couleur.

L'arrière-plan se pilote par une famille de propriétés : background-color pour une couleur unie ; background-image: url("photo.jpg") pour une image ; background-size: cover pour que l'image couvre toute la zone sans se déformer ; background-position: center pour la cadrer ; background-repeat: no-repeat pour empêcher la mosaïque. On peut aussi créer des dégradés sans aucune image : background: linear-gradient(135deg, #0b2545, #1256c4) fond progressivement d'une couleur à l'autre, un effet très utilisé pour les en-têtes modernes.

La typographie donne son caractère à la page. font-family reçoit une liste de polices de repli — font-family: "Segoe UI", Arial, sans-serif — car si la première manque sur l'ordinateur du visiteur, le navigateur essaie la suivante, jusqu'à la famille générique (serif, sans-serif, monospace) qui doit toujours terminer la liste. font-size règle la taille (préférer rem, relatif à la taille racine, pour respecter les réglages d'accessibilité), font-weight la graisse (normal, bold, ou 100 à 900), line-height l'interligne (1.5 est un bon défaut pour la lecture), et text-align, text-decoration, letter-spacing ou text-transform achèvent la mise en forme du texte. Un contraste suffisant entre texte et fond n'est pas une option esthétique : c'est ce qui rend le texte lisible, surtout en plein soleil sur un téléphone.

L'essentiel
Couleurs : noms, hexadécimal (#0b3d91), rgb/rgba (avec transparence), hsl.
Arrière-plans : couleur, image (background-size: cover), ou dégradé linear-gradient.
Typographie : font-family avec replis, font-size en rem, line-height ≈ 1.5, et toujours un contraste lisible.

Exemple — En-tête dégradé et texte soigné

```
body {
font-family: "Segoe UI", Arial, sans-serif;
font-size: 1rem;
line-height: 1.6;
color: #1a1a1a;
}

.banniere {
background: linear-gradient(130deg, #0b2545, #1256c4);
color: #ffffff;
padding: 40px 20px;
text-align: center;
}

.banniere h1 {
font-size: 2.2rem;
letter-spacing: 1px;
text-transform: uppercase;
}
```

À retenir

• rgba = rgb + transparence : parfait pour des fonds doux sans créer une nouvelle couleur.

• background-size: cover évite les images déformées ou répétées en mosaïque.

• Toujours terminer font-family par une famille générique (sans-serif…) en dernier repli.

Exercice

On te donne les styles d'un site : fond de page #f5f7fa, titres #0b2545, texte courant #555555 en « Verdana, Geneva, sans-serif », interligne 1.4. 1) Écris les règles pour body et h1 qui appliquent ces choix. 2) Le fond d'une section doit être le même bleu nuit que les titres, mais à 8 % d'opacité seulement : quelle notation utiliser et quelle valeur écrire ? (Le bleu nuit #0b2545 correspond à rouge 11, vert 37, bleu 69.) 3) Pourquoi la liste de polices se termine-t-elle par sans-serif ?

✔ Corrigé

1) body { background-color: #f5f7fa; color: #555555; font-family: Verdana, Geneva, sans-serif; line-height: 1.4; } et h1 { color: #0b2545; }. 2) Il faut la notation rgba, seule à gérer l'opacité : background-color: rgba(11, 37, 69, 0.08). Attention, le fond derrière la section transparaît à 92 % : le résultat dépend donc de ce qui se trouve dessous — avec un fond de page clair, on obtient un bleu très pâle, exactement l'effet demandé. 3) Verdana ou Geneva peuvent manquer selon le système du visiteur ; la famille générique sans-serif garantit qu'une police sans empattements, proche visuellement, prendra le relais au lieu d'un choix arbitraire du navigateur. Une liste de polices est une chaîne de secours, du plus précis au plus universel.

Leçon 4 — Flexbox : aligner et répartir les éléments

Flexbox résout élégamment un problème autrefois pénible : aligner des éléments sur une ligne (ou une colonne) et répartir l'espace entre eux. On déclare display: flex sur le conteneur, et ses enfants directs deviennent des « items » flexibles disposés le long d'un axe principal, horizontal par défaut. La propriété flex-direction choisit cet axe : row (ligne), column (colonne), ou leurs versions inversées. Toutes les propriétés d'alignement se comprennent par rapport à ces deux axes — principal et perpendiculaire — et c'est la clé de tout le système.

Sur l'axe principal, justify-content répartit les items : flex-start (au début), center, flex-end, space-between (le premier et le dernier collés aux bords, l'espace au milieu), space-around ou space-evenly pour des écarts réguliers. Sur l'axe perpendiculaire, align-items aligne les items : center les centre verticalement dans une ligne — le fameux centrage vertical devient une seule ligne de code —, stretch les étire (valeur par défaut), flex-start et flex-end les collent d'un côté. La propriété gap ajoute un espace régulier entre les items, bien plus simple que des marges à corriger aux extrémités. Un item individuel peut déroger à la règle commune grâce à align-self.

Flexbox sait aussi gérer le manque de place. flex-wrap: wrap autorise les items à passer à la ligne suivante quand ils ne tiennent plus, ce qui fonde les galeries d'images et les listes de cartes adaptatives. Côté items, flex-grow indique la part d'espace libre qu'un item absorbe (deux items de flex-grow 1 et 2 se partageront l'espace en tiers et

deux tiers), flex-shrink sa capacité à rétrécir, et flex-basis sa taille de départ ; le raccourci flex: 1 combine les trois pour des items qui se partagent équitablement la place. Avant Flexbox, centrer un menu ou aligner des cartes demandait des contournements fragiles ; aujourd'hui, une barre de navigation tient en cinq lignes.

L'essentiel
display: flex sur le conteneur. justify-content répartit sur l'axe principal, align-items aligne sur l'axe perpendiculaire, gap espace les items, flex-wrap: wrap autorise le retour à la ligne, et flex: 1 fait partager l'espace libre équitablement.

Exemple — Barre de navigation et cartes en ligne

```
.barre-nav {
display: flex;
justify-content: space-between; /* logo à gauche, menu à droite */
align-items: center; /* centrage vertical */
padding: 12px 20px;
}

.liste-cartes {
display: flex;
flex-wrap: wrap; /* passage à la ligne si besoin */
gap: 16px;
}

.liste-cartes .carte {
flex: 1 1 220px; /* grandit, rétrécit, base 220px */
}
```

À retenir

- Les propriétés d'alignement vont sur le conteneur ; les propriétés flex-grow, flex-shrink, align-self vont sur les items.
- space-between : premier et dernier items aux extrémités, espace libre réparti entre eux.
- flex-direction: column échange les rôles des axes : justify-content agit alors verticalement.

Exercice

Une page contient un pied de page avec trois blocs (contact, liens, réseaux sociaux) qui doivent tenir sur une ligne, répartis avec des écarts égaux, et passer en colonne centrée sur petit écran. 1) Écris le CSS de base du conteneur et des blocs pour la version large. 2) À quoi sert exactement gap dans ce cas, par rapport à des marges sur les blocs ? 3) Quelle propriété changeras-tu dans la media query mobile (vue en leçon 6) pour passer en colonne ?

✔ Corrigé
1) Conteneur : display: flex; justify-content: space-between (ou space-evenly si l'on veut aussi des écarts aux extrémités); gap: 24px; chaque bloc peut recevoir flex: 1 pour occuper un tiers de la largeur. 2) gap crée un espace uniquement ENTRE les items, jamais sur les bords extérieurs : avec des marges, le premier et le dernier bloc auraient aussi un espace contre le bord du conteneur, et il faudrait des exceptions (:first-child, :last-child) pour les retirer. gap évite ces corrections et garde le code lisible. 3) Dans la media query, une seule ligne suffit : flex-direction: column (éventuellement avec align-items: center pour centrer les blocs) — tout le reste du Flexbox continue de fonctionner, les axes tournent simplement de 90 degrés.

Leçon 5 — CSS Grid : mises en page en deux dimensions

Là où Flexbox travaille sur un axe à la fois, CSS Grid compose directement en deux dimensions : des lignes ET des colonnes, comme un tableau de mise en page. On déclare display: grid sur le conteneur, puis on décrit la grille : grid-template-columns: 200px 1fr 1fr crée trois colonnes — une fixe de 200 pixels et deux qui se partagent l'espace restant grâce à l'unité fr (fraction de l'espace libre). Le raccourci repeat(3, 1fr) évite de répéter la même mesure, et gap espace rangées et colonnes d'un seul geste. Les enfants du conteneur se placent alors automatiquement, cellule par cellule, de gauche à droite et de haut en bas.

Le placement peut devenir explicite quand la maquette l'exige. Un item peut couvrir plusieurs cellules : grid-column: 1 / 3 le fait s'étendre de la ligne verticale 1 à la ligne 3 (donc sur deux colonnes), et grid-row fait de même verticalement ; la forme grid-column: span 2 déclare simplement « je couvre deux colonnes » à partir de sa position naturelle. C'est ainsi qu'on construit une page classique : un en-tête sur toute la largeur, une zone de contenu de deux colonnes, une barre latérale, et un pied de page pleine largeur.

Grid offre deux raffinements très appréciés. Les zones nommées, avec grid-template-areas, dessinent la maquette en texte dans le CSS : chaque chaîne décrit une rangée, chaque mot nomme la zone d'une cellule, et les items se placent par grid-area: nom — la grille devient lisible comme un plan. Et pour les galeries au nombre d'éléments variable, repeat(auto-fit, minmax(220px, 1fr)) crée autant de colonnes d'au moins 220 pixels que la largeur permet, sans aucune media query : la grille s'adapte toute seule. Flexbox et Grid ne sont pas concurrents : Grid organise la page, Flexbox aligne le contenu à l'intérieur de chaque zone.

L'essentiel
display: grid + grid-template-columns/rows définissent la grille ; l'unité fr partage l'espace libre. grid-column et grid-row (ou span) font couvrir plusieurs cellules ; grid-template-areas dessine la maquette en zones nommées ; auto-fit + minmax crée des grilles auto-adaptatives.

Exemple — Page classique en zones nommées

```
.page {
display: grid;
grid-template-columns: 1fr 260px;
grid-template-areas:
"entete entete"
"contenu lateral"
"pied pied";
gap: 16px;
}

header { grid-area: entete; }
main { grid-area: contenu; }
aside { grid-area: lateral; }
footer { grid-area: pied; }
```

À retenir

- 1fr = une part de l'espace restant après les tailles fixes : 1fr 2fr partage en tiers / deux tiers.
- Grid = deux dimensions (la page) ; Flexbox = une dimension (le contenu d'une zone). Les deux se combinent.
- repeat(auto-fit, minmax(220px, 1fr)) : une galerie responsive sans media query.

Exercice

Un tableau de bord doit afficher 6 cartes de statistiques : 3 colonnes égales sur écran large, avec 20px d'espace partout. 1) Écris le CSS du conteneur. 2) La première carte est un titre qui doit occuper toute la première rangée : quelle déclaration ajouter à cette carte ? 3) Combien de rangées la grille comptera-t-elle au total, titre compris ?

✔ Corrigé
1) .tableau { display: grid; grid-template-columns: repeat(3, 1fr); gap: 20px; }. Aucune hauteur n'est à déclarer : les rangées se créent automatiquement au fur et à mesure que les items se placent. 2) La carte-titre reçoit grid-column: 1 / -1 (de la première ligne verticale à la dernière — -1 désigne toujours la fin de la grille) ou, de façon équivalente, grid-column: span 3. 3) Le titre prend la rangée 1 à lui seul ; les 6 cartes restantes remplissent les rangées 2 et 3 (3 + 3) : la grille compte donc 3 rangées. Note que si l'on avait mis le titre AVEC les cartes dans le flux sans déclaration particulière, il n'occuperait qu'une cellule et tout serait décalé : le placement explicite est là pour les éléments qui sortent du rang.

Leçon 6 — Design responsive et media queries

Le design responsive consiste à faire en sorte qu'une même page s'affiche agréablement sur un téléphone de 360 pixels de large comme sur un écran de 1920. Deux fondations le rendent possible. D'abord la balise meta viewport, à placer dans le <head> : <meta name="viewport" content="width=device-width, initial-scale=1"> — sans elle, le téléphone affiche la page comme un écran large réduit en miniature, illisible. Ensuite, les dimensions fluides : des largeurs en pourcentages ou en fr, des images bridées par max-width: 100%, et des tailles de texte en rem plutôt que des

mises en page figées au pixel près.

Les media queries sont le mécanisme d'adaptation : @media (max-width: 768px) { … } applique les règles qu'elle contient uniquement si la condition est vraie — ici, un écran d'au plus 768 pixels. À l'intérieur, on corrige ce qui casse sur petit écran : les colonnes passent en une seule, la navigation flex passe en colonne, les tailles de titres diminuent, les marges se resserrent. Les largeurs de rupture courantes sont autour de 480-768 pixels (téléphones et tablettes) et 1024 (petits portables), mais la bonne méthode consiste à rétrécir lentement la fenêtre et à ajouter une rupture là où la mise en page se dégrade réellement.

Deux stratégies s'opposent. Le « desktop first » part de la version large et corrige vers le bas avec des max-width ; le « mobile first », aujourd'hui recommandé, écrit d'abord les styles du téléphone — simples, en une colonne — puis enrichit la mise en page avec des media queries en min-width quand la place le permet. Le mobile first a un avantage concret : le CSS de base est plus léger, et chaque ajout est un progrès visible plutôt qu'une correction. Quelle que soit la stratégie, le test final se fait sur de vrais écrans ou avec les outils de simulation du navigateur (touche F12, mode appareil).

L'essentiel
Responsive = meta viewport obligatoire + dimensions fluides + media queries. @media (max-width: 768px) adapte aux petits écrans ; l'approche mobile first écrit le style du téléphone d'abord, puis enrichit en min-width. Tester en rétrécissant la fenêtre et sur de vrais appareils.

Exemple — De deux colonnes à une seule sur mobile

```
/* Base (mobile first) : une colonne */
.grille-articles {
display: grid;
grid-template-columns: 1fr;
gap: 16px;
}

/* À partir de 768px : deux colonnes */
@media (min-width: 768px) {
.grille-articles {
grid-template-columns: 1fr 1fr;
}
}

/* À partir de 1100px : trois colonnes */
@media (min-width: 1100px) {
.grille-articles {
grid-template-columns: repeat(3, 1fr);
}
}
```

À retenir

• Sans meta viewport, le responsive est impossible : le téléphone « ment » sur sa largeur.

• max-width dans une media query = « écrans jusqu'à… » ; min-width = « écrans à partir de… ».

• Une image responsive : max-width: 100%; height: auto; — elle ne dépasse jamais son conteneur et garde ses proportions.

Exercice

Un site a été conçu « desktop first » avec une navigation horizontale et un texte de 18px. Sur téléphone, la navigation déborde. 1) Écris la media query qui, sous 700px, passe la navigation (classe .menu, conteneur flex) en colonne et réduit le texte du corps à 16px. 2) Le menu passe en colonne mais reste trop large : les liens ont un padding horizontal de 30px. Que peux-tu corriger dans la même media query ? 3) Pourquoi ne faut-il pas simplement cacher le menu sur mobile avec display: none ?

✔ Corrigé
1) @media (max-width: 700px) { .menu { flex-direction: column; } body { font-size: 16px; } }. 2) Les paddings font partie de la largeur totale des liens : dans la même requête, réduire l'espace intérieur — .menu a { padding: 10px 12px; } — et, si le conteneur l'exige, passer align-items: stretch ou center selon l'effet voulu. Les débordements mobiles viennent le plus souvent de paddings et de largeurs fixes hérités de la version bureau, pas des textes eux-mêmes. 3) display: none supprime l'accès à la navigation : les visiteurs mobiles ne pourraient plus changer de page. Le responsive adapte la présentation, il ne retire pas les fonctions — un menu en colonne, ou replié derrière un bouton, reste un menu utilisable.

Leçon 7 — Transitions et animations CSS

Une transition adoucit un changement d'état : au lieu qu'un bouton passe brutalement d'un fond bleu à un fond doré au survol, la couleur glisse progressivement. On la déclare sur l'état de départ : transition: background-color 0.3s ease; signifie « quand background-color change, anime le passage en 0,3 seconde avec un ralentissement en fin de course ». Les propriétés animables sont nombreuses — couleurs, tailles, marges, opacity, transform — et le raccourci transition: all 0.3s ease les couvre toutes, au prix d'un contrôle moindre. Les courbes d'accélération (ease, linear, ease-in-out) règlent le caractère du mouvement.

La propriété transform déplace ou déforme un élément sans toucher à la mise en page autour : translate(10px, -5px) le décale, scale(1.05) l'agrandit de 5 %, rotate(3deg) le pivote légèrement. Combinée aux transitions, elle produit l'effet « carte qui se soulève » des sites modernes : au survol, transform: translateY(-4px) et une ombre plus marquée donnent une impression de relief. Point important : transform n'écarte pas les éléments voisins — la carte déborde visuellement par-dessus, sans casser la grille — ce qui la rend parfaite pour les micro-effets.

Quand le mouvement doit se répéter ou suivre plusieurs étapes, on passe aux animations par images clés. @keyframes nom décrit les étapes (from/to, ou des pourcentages 0%, 50%, 100%), et la propriété animation applique la scène : animation: apparition 0.6s ease both; lance « apparition » une fois, en gardant l'état final grâce à both. On règle la durée, le nombre de répétitions (infinite pour une

boucle), et la direction. La sobriété reste la règle professionnelle : des animations courtes, qui guident l'œil (faire apparaître un contenu, signaler un survol), jamais un spectacle permanent — et l'on respecte les visiteurs sensibles au mouvement grâce à la media query prefers-reduced-motion, qui permet de réduire ou couper les animations.

L'essentiel
transition anime un changement d'état (survol, focus) en douceur ; transform (translate, scale, rotate) bouge l'élément sans perturber la mise en page ; @keyframes + animation créent des mouvements en plusieurs étapes ou en boucle. Toujours rester sobre et respecter prefers-reduced-motion.

Exemple — Bouton fluide et carte qui se soulève

```
.bouton {
background-color: #0b3d91;
color: #ffffff;
transition: background-color 0.3s ease, transform 0.3s ease;
}

.bouton:hover {
background-color: #c9a227;
transform: translateY(-2px);
}

@keyframes apparition {
from { opacity: 0; transform: translateY(12px); }
to { opacity: 1; transform: translateY(0); }
}

.carte {
animation: apparition 0.6s ease both;
}
```

À retenir

- La transition se déclare sur l'état normal, pas sur :hover — sinon le retour serait brutal.
- transform ne modifie pas la place des voisins : idéal pour les effets de survol.
- Une bonne animation est courte (0,2 à 0,6 s), a une raison d'être, et peut être désactivée (prefers-reduced-motion).

Exercice

Un lien de menu doit, au survol, changer de couleur en une demi-seconde et se souligner progressivement par un léger grossissement du texte simulé par scale. 1) Écris le CSS de base du lien et de son :hover avec transition. 2) Pourquoi déclarer la transition sur le lien de base et pas seulement sur :hover ? 3) Un visiteur a activé « réduire les animations » dans son système : écris la media query qui coupe la transition pour lui.

✔ Corrigé
1) .menu a { color: #ffffff; transition: color 0.5s ease, transform 0.5s ease; display: inline-block; } .menu a:hover { color: #c9a227; transform: scale(1.08); }. Le display: inline-block est nécessaire car transform est ignoré sur les éléments inline purs comme les liens par défaut. 2) La transition décrite sur :hover ne jouerait qu'à l'entrée du survol ; en sortant, le navigateur reviendrait à l'état de base sans aucune transition déclarée, donc instantanément — un effet « doux dans un sens, sec dans l'autre ». Déclarée sur l'élément de base, elle s'applique dans les deux directions. 3) @media (prefers-reduced-motion: reduce) { .menu a { transition: none; } } — on peut généraliser à toute la page avec * { animation: none; transition: none; } dans cette requête : l'information reste accessible, seul le mouvement disparaît.

Leçon 8 — Projet : reproduire une maquette web moderne

Reproduire une maquette — une image de la page à obtenir — est l'exercice qui transforme des notions séparées en savoir-faire. La méthode compte plus que la vitesse. Étape 1, analyser : regarder la maquette en plissant les yeux pour repérer les grands blocs (en-tête, bannière, sections, cartes, pied de page) et la palette de couleurs. Étape 2, structurer : écrire le HTML sémantique complet sans aucun style, avec des classes nommées par rôle (.hero, .section-cours, .carte-cours) plutôt que par apparence. Étape 3, poser les fondations CSS : la réinitialisation douce (* { box-sizing: border-box; margin: 0; }), les variables de couleurs, la typographie générale. On ne passe à l'étape suivante que lorsque la structure tient debout.

Étape 4, habiller bloc par bloc, du haut vers le bas : d'abord les fonds et les textes, puis les dispositions — Grid pour la structure de la page, Flexbox pour l'intérieur des barres et des rangées de cartes. Les variables CSS, déclarées dans :root (--bleu: #0b2545; --or: #c9a227;), centralisent la charte : un changement de couleur se fait ensuite en un seul endroit, et var(--bleu) remplace les valeurs recopiées. Étape 5, rendre responsive : rétrécir la fenêtre, ajouter les media queries là où ça casse, vérifier sur téléphone. Étape 6, polir : survols, transitions, espacements réguliers — et comparer côte à côte avec la maquette, en corrigeant les écarts les plus visibles d'abord.

Notre maquette d'entraînement est la page d'accueil d'une bibliothèque universitaire : en-tête bleu nuit avec logo et navigation, grande bannière dégradée avec un titre et un bouton doré, une rangée de trois cartes de filières (Informatique, BBA, Droit), une section « cours populaires » en grille, et un pied de page sombre en trois colonnes. Chaque élément correspond à une leçon du cours : sélecteurs et cascade pour les styles de base, modèle de boîte pour les cartes, Flexbox pour la navigation et le pied de page, Grid pour les cours, media queries pour le mobile, transitions pour les survols. Si un bloc résiste, la bonne question n'est pas « quel code copier » mais « quelle leçon traite ce problème ».

L'essentiel
Méthode en 6 étapes : analyser la maquette en blocs, écrire le HTML sémantique, poser les fondations (box-sizing, variables, typographie), habiller bloc par bloc (Grid pour la page, Flexbox pour l'intérieur), adapter en responsive, puis polir (survols, transitions). Les variables CSS dans :root centralisent la charte graphique.

Exemple — Fondations et bannière de la maquette

```
:root {
--bleu-nuit: #0b2545;
--bleu: #1256c4;
--or: #c9a227;
--fond: #f5f7fa;
}

* { box-sizing: border-box; margin: 0; }

body {
font-family: "Segoe UI", Arial, sans-serif;
background-color: var(--fond);
line-height: 1.6;
}

.hero {
background: linear-gradient(130deg, var(--bleu-nuit), var(--bleu));
color: #ffffff;
text-align: center;
padding: 60px 20px;
}

.hero .cta {
background-color: var(--or);
color: #1a1a1a;
padding: 12px 24px;
border-radius: 999px;
text-decoration: none;
transition: background-color 0.3s ease;
}
```

À retenir

- On ne style jamais avant d'avoir une structure HTML complète et des classes bien nommées.
- Variables CSS : toute couleur utilisée plus de deux fois mérite une variable dans :root.
- L'ordre des finitions : comparer à la maquette, corriger les gros écarts, puis les détails.

Exercice

Tu dois reproduire la rangée des trois cartes « filières » de la maquette : trois cartes égales côte à côte sur écran large, chacune avec une icône, un titre et un texte, fond blanc, coins arrondis, ombre légère ; sur téléphone, elles s'empilent. 1) Choisis entre

Flexbox et Grid pour cette rangée et justifie en une phrase. 2) Écris le HTML du conteneur et d'une carte (les deux autres sont identiques en structure). 3) Écris le CSS complet de la version large et la media query mobile.

✔ Corrigé
1) Les deux systèmes conviennent ; Grid est ici le plus direct (trois colonnes égales annoncées en une ligne) — Flexbox avec flex: 1 sur les cartes serait tout aussi valable. 2) HTML : <section class="filieres"> <article class="carte-filiere"> □ <h3>Informatique</h3> Algorithmique, programmation, réseaux… </article> … </section> — un article par carte, la structure reste claire même sans CSS. 3) CSS large : .filieres { display: grid; grid-template-columns: repeat(3, 1fr); gap: 20px; padding: 40px 20px; } .carte-filiere { background: #ffffff; border-radius: 14px; padding: 24px; box-shadow: 0 8px 20px rgba(11, 37, 69, 0.10); text-align: center; }. Mobile : @media (max-width: 768px) { .filieres { grid-template-columns: 1fr; } } — une seule ligne change, les cartes et leurs styles restent intacts : c'est la récompense d'une base bien construite.

Travaux dirigés (TD)

TD 1 — Habiller la page HTML du cours précédent

Reprends la page HTML produite au cours INF-L1-04 (page d'accueil d'un site personnel : en-tête avec navigation, section de présentation, section « mes projets » avec trois projets, pied de page). Crée une feuille externe styles.css qui : 1) applique box-sizing: border-box à tous les éléments et supprime la marge par défaut du body ; 2) centre le contenu dans un conteneur de 1000px maximum avec des marges latérales automatiques ; 3) met l'en-tête en bleu nuit avec liens blancs qui deviennent dorés au survol (transition 0,3 s) ; 4) dispose les trois projets en ligne avec Flexbox et 16px d'espace entre eux, chaque projet dans une carte blanche à coins arrondis et padding 20px ; 5) sous 700px, fait passer les projets en colonne. Livrable : le fichier CSS complet, commenté par sections.

✔ Corrigé du TD 1
Points de contrôle du corrigé type : la réinitialisation ouvre le fichier (* { box-sizing: border-box; } body { margin: 0; font-family: …; }). Le conteneur : .conteneur { max-width: 1000px; margin: 0 auto; padding: 0 16px; } — le padding latéral évite que le texte touche les bords sur mobile. En-tête : header { background-color: #0b2545; } nav a { color: #ffffff; text-decoration: none; transition: color 0.3s ease; } nav a:hover { color: #c9a227; }. Projets : .projets { display: flex; gap: 16px; } .projet { flex: 1; background: #ffffff; border-radius: 12px; padding: 20px; } — flex: 1 donne trois colonnes égales sans calculer de pourcentages. Media query : @media (max-width: 700px) { .projets { flex-direction: column; } }. Erreurs fréquentes à traquer : marges du body oubliées (décalage blanc autour de l'en-tête), transition placée sur :hover seulement (retour brutal), et colonnes qui débordent parce que box-sizing n'a pas été appliqué.

TD 2 — Déboguer une feuille de style qui « ne marche pas »

Un camarade te montre sa page : les titres devaient être bleus mais ils sont noirs ; le bouton « Valider » devait être centré mais il reste à gauche ; la carte de 200px de large « dépasse » de son conteneur de 200px. Son CSS contient : h2.titre { color: blue; } alors que ses titres portent class="Titre" ; .bouton { margin: 0 auto; } sur un lien <a> sans display particulier ; .carte { width: 200px; padding: 20px; border: 5px solid; } sans box-sizing. Pour chaque symptôme : donne la cause exacte et la correction minimale, en citant le concept du cours concerné (leçons 1 et 2).

✔ Corrigé du TD 2
1) Titres noirs : la casse compte en CSS — le sélecteur .titre ne correspond pas à la classe "Titre" du HTML ; la règle ne vise donc rien. Correction : aligner les deux orthographes (de préférence tout en minuscules, par convention). Concept : le sélecteur de classe doit correspondre exactement à l'attribut class. 2) Bouton non centré : margin: 0 auto ne centre que les éléments block ayant une largeur définie ; un <a> est inline par défaut, les marges automatiques n'ont pas d'effet. Correction : .bouton { display: block; width: 200px; margin: 0 auto; } ou placer le lien dans un conteneur en text-align: center avec display: inline-block sur le lien. Concept : la propriété display décide du comportement de la boîte. 3) Carte qui déborde : en content-box, la largeur totale vaut 200 + 2×20 + 2×5 = 250px. Correction : .carte { box-sizing: border-box; } (ou l'appliquer globalement avec *) : la largeur redevient 200px tout compris. Concept : le modèle de boîte et box-sizing. Morale du TD : neuf pannes CSS sur dix sont une question de sélecteur qui ne vise rien, de display inadapté, ou de calcul de boîte — dans cet ordre, vérifie ces trois points avant de réécrire ton code.

Travaux pratiques (TP)

TP 1 — Construire et adapter la maquette « Bibliothèque universitaire »

Sur ton poste, crée les fichiers index.html et style.css et reproduis la maquette de la leçon 8 : en-tête bleu nuit (logo à gauche, navigation à droite, Flexbox), bannière dégradée avec titre et bouton doré, rangée de trois cartes filières (Grid), grille de six cours (Grid auto-fit minmax), pied de page sombre en trois colonnes (Flexbox). Ajoute un effet de survol sur chaque carte (montée de 4px + ombre renforcée, transition 0,3 s) et rends l'ensemble responsive : une colonne pour les cartes et le pied de page sous 768px. Attendus : les deux fichiers, une capture écran large et une capture en largeur 375px (mode appareil des outils de développement, touche F12), et la liste des trois difficultés rencontrées avec la façon dont tu les as résolues.

✔ Attendus du TP 1
Attendus techniques : un :root avec les variables de la charte (--bleu-nuit, --bleu, --or, --fond) utilisées partout via var() ; box-sizing global ; grille des cours en repeat(auto-fit, minmax(220px, 1fr)) — si tu as écrit des media queries pour cette grille, compare avec cette solution en une ligne ; cartes avec transition: transform 0.3s ease, box-shadow 0.3s ease et :hover { transform: translateY(-4px); }. Les difficultés qui reviennent le plus souvent, et leurs remèdes : navigation qui passe à la ligne trop tôt (manque flex-wrap ou media query trop tardive), images qui déforment les cartes (max-width: 100% et hauteurs d'images harmonisées), pied de page collé aux bords sur mobile (padding latéral du conteneur oublié). La capture 375px doit montrer : navigation lisible, aucune barre de défilement horizontale, cartes empilées avec leurs espacements conservés.

Exercices corrigés supplémentaires

Exercice 1

Sans regarder le cours, écris la règle qui centre horizontalement et verticalement un texte dans une boîte de 300px de haut portant la classe .boite, en utilisant Flexbox. Explique le rôle de chaque propriété.

✔ Corrigé
.boite { display: flex; justify-content: center; align-items: center; height: 300px; }. display: flex transforme la boîte en conteneur flexible ; justify-content: center centre le contenu sur l'axe principal (horizontal en direction row) ; align-items: center centre sur l'axe perpendiculaire (vertical). La hauteur fixe est indispensable : sans elle, la boîte épouse la hauteur du texte et le « centrage vertical » n'a aucun espace où s'exprimer — c'est l'oubli le plus fréquent de cet exercice.

Exercice 2

Un élément a la classe .promo et l'identifiant #promo-principale. Trois règles existent : .promo { background-color: gold; }, #promo-principale { background-color: silver; }, div.promo { background-color: bronze; }. De quelle couleur sera le fond de l'élément, et pourquoi ? Classe les trois règles par spécificité croissante.

✔ Corrigé
Le fond sera silver. Spécificités croissantes : .promo (une classe) < div.promo (une classe + une balise) < #promo-principale (un identifiant). L'identifiant est le sélecteur le plus spécifique des trois : sa règle l'emporte quelle que soit sa position dans le fichier. La règle div.promo battrait .promo malgré les apparences, car ajouter la balise augmente la spécificité. En pratique professionnelle, on évite de surenchérir : si tu dois écrire des sélecteurs de plus en plus longs pour « gagner », c'est le signe qu'une classe mieux nommée ou une réorganisation de la feuille s'impose.

Exercice 3

Écris le CSS d'une galerie de photos : autant de colonnes que possible, chaque photo d'au moins 180px de large, 12px d'espace entre elles, les images occupant toute la largeur de leur cellule sans déformation (hauteur fixe de 140px, recadrage propre). Aucune media query n'est autorisée.

✔ Corrigé
.galerie { display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr)); gap: 12px; } .galerie img { width: 100%; height: 140px; object-fit: cover; }. Le couple auto-fit/minmax crée les colonnes et les fait passer à la ligne automatiquement quand la place manque — c'est la grille auto-adaptative de la leçon 5. object-fit: cover fait pour les images ce que background-size: cover fait pour les fonds : remplir la zone en recadrant ce qui dépasse, sans jamais étirer ni écraser le sujet. Sans object-fit, imposer width et height déformerait chaque photo aux proportions différentes.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Dans la règle p { color: blue; }, « color » est…

A) Le sélecteur

B) Une propriété

C) Une valeur

2. Quel sélecteur vise tous les éléments portant class="carte" ?

A) #carte

B) carte

C) .carte

3. En mode content-box (par défaut), une boîte de width: 100px, padding: 10px, border: 2px mesure…

A) 100px de large au total

B) 124px de large au total

C) 114px de large au total

4. Avec box-sizing: border-box, width: 200px signifie…

A) 200px de contenu, padding en plus

B) 200px pour la largeur totale, padding et bordure inclus

C) 200px de padding

5. Pour répartir des items flex avec le premier et le dernier collés aux bords, on utilise…

A) justify-content: space-between

B) align-items: space-between

C) flex-direction: space-between

6. Quelle propriété permet aux items flex de passer à la ligne quand la place manque ?

A) flex-flow: nowrap

B) overflow: wrap

C) flex-wrap: wrap

7. Dans grid-template-columns: 1fr 2fr, les colonnes se partagent l'espace libre en…

A) Moitié-moitié

B) Un tiers / deux tiers

C) 1 pixel et 2 pixels

8. Une media query @media (max-width: 600px) s'applique quand…

A) L'écran fait exactement 600px

B) L'écran fait au moins 600px

C) L'écran fait 600px ou moins

9. Pour qu'une image ne dépasse jamais de son conteneur tout en gardant ses proportions, on écrit…

A) img { width: 100%; } seulement

B) img { max-width: 100%; height: auto; }

C) img { height: 100%; }

10. La transition d'un effet de survol doit être déclarée…

A) Uniquement dans la règle :hover

B) Sur l'élément dans son état normal

C) Dans le fichier HTML

Corrigés du QCM

1. Réponse B — La structure d'une déclaration est propriété: valeur ; ici la propriété est color et la valeur est blue. Le sélecteur est p, devant les accolades.

2. Réponse C — Le point désigne une classe ; le dièse désigne un identifiant ; le nom seul désigne une balise.

3. Réponse B — 100 + 10 + 10 (padding des deux côtés) + 2 + 2 (bordures) = 124px. C'est le piège que box-sizing: border-box élimine.

4. Réponse B — border-box fait entrer padding et bordure DANS la largeur déclarée : le contenu utile se réduit d'autant.

5. Réponse A — La répartition sur l'axe principal relève de justify-content ; space-between met le premier et le dernier item aux extrémités. align-items gère l'axe perpendiculaire et n'a pas de valeur space-between.

6. Réponse C — flex-wrap: wrap autorise le retour à la ligne ; par défaut (nowrap), les items rétrécissent sur une seule ligne.

7. Réponse B — L'unité fr est proportionnelle : 1fr reçoit une part, 2fr en reçoit deux, soit un tiers et deux tiers de l'espace disponible.

8. Réponse C — max-width définit un plafond : la règle vaut pour les écrans jusqu'à 600px inclus — typiquement les téléphones.

9. Réponse B — max-width: 100% borne l'image à la largeur du conteneur et height: auto conserve le rapport largeur/hauteur d'origine.

10. Réponse B — Déclarée sur l'état normal, la transition joue à l'aller comme au retour du survol ; déclarée seulement sur :hover, le retour serait instantané.

Résumé — les notions importantes

• Une règle CSS associe un sélecteur (balise, .classe, #identifiant, descendants, pseudo-classes) à des déclarations propriété: valeur ; la cascade tranche les conflits par la spécificité puis par l'ordre.

• Chaque élément est une boîte : contenu, padding, bordure, marge. box-sizing: border-box fait de width la largeur totale — à appliquer partout.

• Couleurs en hexadécimal, rgb/rgba ou hsl ; fonds en couleur, en image (cover) ou en dégradé ; typographie en font-family à replis, tailles en rem, interligne généreux.

• Flexbox aligne sur un axe : justify-content pour l'axe principal, align-items pour l'axe perpendiculaire, gap pour les écarts, flex-wrap pour les retours à la ligne.

• CSS Grid compose en deux dimensions : colonnes en fr, placement par lignes ou zones nommées, et grilles auto-adaptatives avec auto-fit et minmax.

• Responsive : meta viewport, dimensions fluides, media queries aux points de rupture réels ; démarche mobile first recommandée.

• Transitions et @keyframes animent en douceur (transform, opacity, couleurs) — avec sobriété et respect de prefers-reduced-motion. Méthode projet : analyser, structurer, fonder, habiller, adapter, polir.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Syntaxe CSS : sélecteurs, propriétés et valeurs
Le CSS externe (fichier .css relié par <link>) est la norme : un fichier pour tout le site. ; .classe se réutilise sur autant d'éléments qu'on veut ; #identifiant ne doit exister qu'une fois par page. ; Spécificité : identifiant > classe > balise ; à égalité, la dernière règle gagne.

Fiche 2 — Le modèle de boîte : marges, bordures et padding
Ordre des couches, de l'intérieur : contenu → padding → bordure → marge. ; margin: 0 auto centre horizontalement une boîte block de largeur définie. ; Les marges verticales voisines fusionnent : l'écart vaut la plus grande des deux, pas leur somme.

Fiche 3 — Couleurs, arrière-plans et typographie
rgba = rgb + transparence : parfait pour des fonds doux sans créer une nouvelle couleur. ; background-size: cover évite les images déformées ou répétées en mosaïque. ; Toujours terminer font-family par une famille générique (sans-serif…) en dernier repli.

Fiche 4 — Flexbox : aligner et répartir les éléments
Les propriétés d'alignement vont sur le conteneur ; les propriétés flex-grow, flex-shrink, align-self vont sur les items. ; space-between : premier et dernier items aux extrémités, espace libre réparti entre eux. ; flex-direction: column échange les rôles des axes : justify-content agit alors verticalement.

Fiche 5 — CSS Grid : mises en page en deux dimensions
1fr = une part de l'espace restant après les tailles fixes : 1fr 2fr partage en tiers / deux tiers. ; Grid = deux dimensions (la page) ; Flexbox = une dimension (le contenu d'une zone). Les deux se combinent. ; repeat(auto-fit, minmax(220px, 1fr)) : une galerie responsive sans media query.

Fiche 6 — Design responsive et media queries
Sans meta viewport, le responsive est impossible : le téléphone « ment » sur sa largeur. ; max-width dans une media query = « écrans jusqu'à… » ; min-width = « écrans à partir de… ». ; Une image responsive : max-width: 100%; height: auto; — elle ne dépasse jamais son conteneur et garde ses proportions.

Fiche 7 — Transitions et animations CSS
La transition se déclare sur l'état normal, pas sur :hover — sinon le retour serait brutal. ; transform ne modifie pas la place des voisins : idéal pour les effets de survol. ; Une bonne animation est courte (0,2 à 0,6 s), a une raison d'être, et peut être désactivée (prefers-reduced-motion).

Fiche 8 — Projet : reproduire une maquette web moderne
On ne style jamais avant d'avoir une structure HTML complète et des classes bien nommées. ; Variables CSS : toute couleur utilisée plus de deux fois mérite une variable dans :root. ; L'ordre des finitions : comparer à la maquette, corriger les gros écarts, puis les détails.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Apprenez le CSS en 1 heure ! Cours Complet Débutant 2026

Chaîne : Comment Coder · Langue : Français · Niveau : Débutant

Un cours complet et récent qui parcourt les fondations du CSS en vidéo : écrire ses premières règles, cibler les éléments et organiser sa feuille de style. À regarder en ouverture du cours, puis à revisiter leçon par leçon.

https://www.youtube.com/watch?v=05mkyEm3Jiw

Leçon 2 — Cours Complet HTML CSS - Tutoriel pour Débutants et Confirmés [Partie 1/3]

Chaîne : Pierre Giraud · Langue : Français · Niveau : Débutant

Première partie d'un cours complet très progressif, utile pour consolider les bases pas à pas : la façon dont le navigateur calcule l'espace de chaque élément y est expliquée en détail. Idéal en accompagnement des leçons 1 et 2, notamment le modèle de boîte.

https://www.youtube.com/watch?v=8FqZZrbnwkM

Leçon 4 — Structurer son site avec Flexbox (Tuto débutant)

Chaîne : Graven - Développement · Langue : Français · Niveau : Débutant

Un tutoriel clair qui montre comment structurer une page entière avec Flexbox : aligner, répartir et organiser les blocs sans contournements. Parfait pour voir la leçon 4 en action avant de refaire l'exercice de la barre de navigation.

https://www.youtube.com/watch?v=2qA4mobfcK0

Leçon 5 — Débuter avec CSS Grid - Partie 1

Chaîne : SwebDev · Langue : Français · Niveau : Débutant

Une première partie consacrée aux fondamentaux de CSS Grid : créer la grille, définir colonnes et rangées et placer les éléments. Elle prolonge directement la leçon 5 ; la suite de la série permet d'aller plus loin une fois les bases acquises.

https://www.youtube.com/watch?v=1_SW9DO4tKs

Leçon 6 — Les Bases du Responsive avec les @MediaQueries | HTML - CSS

Chaîne : Cédric Design · Langue : Français · Niveau : Débutant à intermédiaire

Les media queries expliquées simplement : écrire les conditions, choisir les points de rupture et adapter concrètement une mise en page aux téléphones. Le complément parfait de la leçon 6 avant d'attaquer le projet final.

https://www.youtube.com/watch?v=BdzKNrxLiSs

Sources et références

- Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.
- MDN Web Docs (developer.mozilla.org/fr) — la documentation de référence du CSS : chaque propriété y est décrite avec des exemples interactifs.
- Documentation et guides visuels Flexbox et Grid sur MDN (fr) — pour réviser les alignements avec des schémas.
- Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.